Guía de REA: instalación y funcionamiento

Oct 21, 2026 · @Chantal De Almeida Palmucci

Qué es REA

REA (*Reverse Engineer Anything*) es una herramienta gratuita y de código abierto (licencia MIT) que deja a un agente de IA, como Claude Code, **ver cómo funciona por dentro una aplicación de la que no tienes el código fuente**. Se publica en github.com/morluto/rea, en rea.tools y como paquete de npm `rea-agents`; la versión que instalamos fue la 6.5.0.

La idea es: ves una función que te gusta en una app, le pides a tu agente que la investigue y él te explica cómo está hecha, te enseña las pruebas y puede construir algo parecido en tu proyecto.

REA tiene dos formas de uso:

- **Servidor MCP:** el agente lo usa como un conjunto de herramientas propias (más de 200), sin que tú escribas comandos.
- **Comando `rea` en la terminal:** los mismos análisis, lanzados a mano.

Todo el análisis ocurre en tu ordenador; no sube la aplicación a ningún sitio.

Requisitos

Lo único obligatorio es **Node.js con npm**; lo demás depende de qué quieras analizar.

Qué	¿Obligatorio?	Detalle
Node.js	Sí	Versión 22.x (≥ 22.19), 24.x (≥ 24.11) o 26 en adelante. En tu equipo había la 24.21.
npm	Sí	Viene con Node.js. En tu equipo, la 11.19.
Un agente compatible con MCP	Para usarlo con IA	Claude Code, Codex, Cursor, Gemini CLI, Grok Build y otros.
Hopper, Ghidra o IDA	Solo para programas nativos (.exe, .dll, apps de Mac)	REA no los instala, salvo Hopper si lo apruebas. Ghidra necesita la variable `GHIDRA_INSTALL_DIR`.

Qué	¿Obligatorio?	Detalle
JADX + JDK	Solo para APK de Android	Se indica con `REA_JADX_MCP_JAR`.
Navegador tipo Chrome	Solo para analizar webs	

Para apps de JavaScript o Electron (VS Code, Slack, Notion…) y para .NET no hace falta ningún motor extra.

Instalación paso a paso (Windows + Claude Code)

Son seis pasos cortos: comprobar Node.js, instalar el comando, ver qué va a cambiar, aplicarlo, reiniciar Claude y comprobar.

1. **Comprueba Node.js.** En una terminal:

   ```
   node --version
   ```

   Tiene que salir 22.19 o superior (o 24.11+).
2. **Instala el comando `rea`:**

   ```
   npm install --global rea-agents
   ```

   Comprueba que funciona con `rea --version`. Verás un aviso de que npm no ejecutó el script de instalación de **Frida**: solo hace falta para espiar programas mientras se ejecutan. Si lo quieres, instala con `npm install -g --allow-scripts=frida rea-agents`.
3. **Mira en seco qué va a tocar** (no cambia nada):

   ```
   rea setup --client claude_code --dry-run
   ```

   El nombre del cliente lleva guion bajo: `claude_code` (con guion da error). En tu caso planificó dos cosas: añadir el servidor MCP en `C:\Users\chant\.claude.json` (guardando antes una copia en `.claude.json.rea.backup`) e instalar la skill `reverse-engineer-anything` en `C:\Users\chant\.claude\skills\`.
4. **Aplícalo tú** desde una terminal normal:

   ```
   rea setup --client claude_code --yes
   ```

   Claude no puede hacerlo por ti en modo automático: el sistema bloquea que modifique su propia configuración.

5. **Reinicia Claude** (cierra y abre la app o la sesión). Las herramientas de REA solo aparecen en sesiones nuevas.
6. **Comprueba** que quedó registrado:

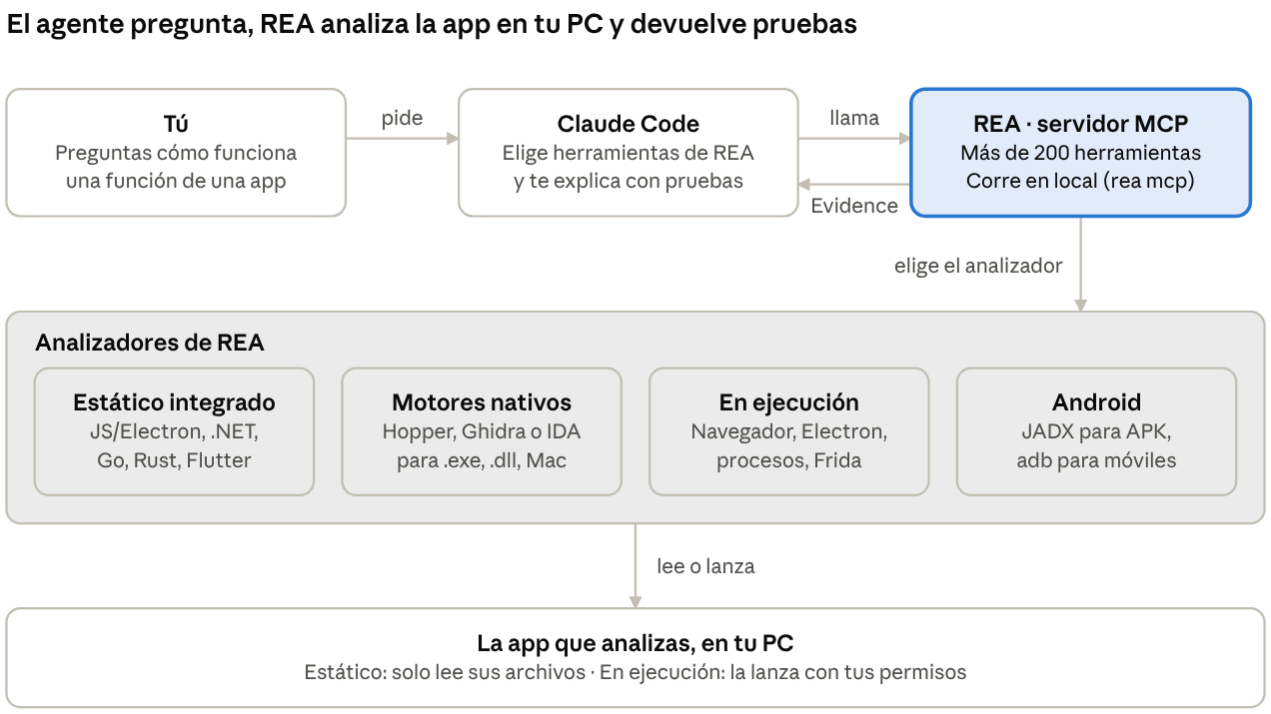
   ```
   claude mcp get rea
   ```

   Y para ver qué motores opcionales faltan: `rea doctor`.

Otros nombres de cliente válidos: `claude_desktop`, `codex`, `cursor`, `gemini_cli`, `windsurf`, `vscode`, `copilot_cli`, entre otros. `rea setup --all-detected` configura todos los que encuentre.

Cómo funciona por dentro

REA es un intermediario: el agente le pide un análisis por MCP, REA usa el analizador que toca y devuelve los hallazgos con sus pruebas.



cómo funciona REA · del agente a la app y vuelta

MCP (*Model Context Protocol*) es el estándar con el que un agente usa herramientas externas. Claude arranca `rea mcp` como un proceso aparte en tu PC y le habla por su entrada y salida estándar.

Evidence es lo que distingue a REA: cada resultado es un registro con un identificador (`ev_…`) que guarda qué se analizó, su huella SHA-256, dónde está cada hallazgo y sus límites. Cada conclusión viene marcada como:

- **Observado:** está literalmente en los archivos.
- **Derivado:** sale de combinar varias observaciones.
- **Inferido:** es una deducción probable.
- **Desconocido:** REA no pudo averiguarlo, y lo dice en vez de inventarlo.

Primero el resumen. Con apps grandes, REA guarda el análisis completo y devuelve solo un resumen. El agente pide luego partes concretas por su identificador, sin repetir el análisis.

Estático o en ejecución. El análisis estático lee el código sin ejecutarlo, así que no prueba qué pasa al usar la app. Para eso están las capturas en ejecución, que sí la lanzan.

Todo lo que puede analizar

REA cubre 23 tipos de objetivo; los que más te pueden servir son las apps de escritorio en JavaScript/Electron y las webs, que no necesitan nada extra.

Objetivo	Qué te devuelve	Qué necesita
Apps JavaScript / Electron	Módulos, imports, source maps, rutas, mensajes IPC entre procesos	Nada extra
Webs	Estructura, scripts, tráfico de red, capturas	Navegador tipo Chrome
Capturas de red guardadas (HAR, SAZ)	Peticiones, respuestas y dónde se originan	Nada extra
Programas nativos (.exe, .dll, Mac)	Pseudocódigo, ensamblador, cadenas, símbolos, llamadas	Hopper, Ghidra o IDA
Ensamblados .NET	Metadatos, instrucciones CIL, dependencias nativas	Nada extra
Binarios de Go y Rust	Versión del compilador, módulos y dependencias	Nada extra
APK de Android	Manifiesto, clases, métodos descompilados	JADX y un JDK
Dispositivos Android	Paquetes, procesos, logs, capturas de pantalla	adb
Apps Flutter	Hash del snapshot de Dart y versión del motor	Nada extra
WebAssembly	Secciones, imports/exports, WAT	WABT 1.0.42

Objetivo	Qué te devuelve	Qué necesita
Bases de datos SQLite	Tablas, columnas, índices, filas de ejemplo	Node.js 24.15+
Contratos EVM (bytecode)	Selectores de funciones y argumentos inferidos	Solo Linux x64
Firmware	Regiones y archivos extraídos	Binwalk o Unblob en Linux
Capturas de paquetes (pcap)	Endpoints, DNS, TLS, peticiones HTTP	TShark
Comportamiento de procesos	Salida, interacciones y archivos tocados	Linux o macOS

El análisis estático de JavaScript y .NET solo lee archivos, sin ejecutar la app. Las capturas en ejecución sí la lanzan, con tus permisos de usuario.

Cómo usarlo

Lo normal es pedirle las cosas al agente en lenguaje natural; la terminal es para análisis sueltos o para automatizar.

Desde Claude, una vez instalado y reiniciado:

```
Entiende cómo funciona la búsqueda en la app de Notion, enséñame las pruebas y construye algo parecido en mi proyecto.
```

Cambia la app y la función por lo que te interese. El agente elige solo la herramienta de REA adecuada según el tipo de app.

Desde la terminal, comandos útiles:

Comando	Para qué
`rea --help`	Lista todos los comandos
`rea doctor`	Dice qué está listo y qué motor falta
`rea analyze-javascript-application "C:/ruta/app" --json`	Reconstruye una app Electron o una carpeta de JavaScript
`rea analyze`	Resumen general de una app

Comando	Para qué
`rea decompile`	Lee una parte de una app como código (necesita motor nativo)
`rea capabilities`	Qué hace cada motor y qué efectos tiene

Con `npx` funciona sin instalar nada: `npx -y rea-agents@latest <comando>`.

Ejemplo real: cómo analizamos VS Code

Con REA vimos que VS Code 1.140 es una app Electron repartida en varios procesos, y con eso construimos el clon Mini Code.

1. **Localizar la app.** El código de VS Code está en `AppData\Local\Programs\Microsoft VS Code\<versión>\resources\app\out` (69 MB de JavaScript).
2. **Analizar solo los archivos de arranque.** Se le pasó a `analyze_javascript_application` una lista corta (`main.js`, `mainImpl.js`, el preload, `workbench.js`, `bootstrap-fork.js`) con `detail: summary`, para no traer 5.931 archivos de golpe.
3. **Seguir el rastro del preload** con `trace_application_feature`.

Qué salió:

Hallazgo	Tipo
El preload expone `window.vscode` con 6 piezas: `context`, `ipcMessagePort`, `ipcRenderer`, `process`, `webFrame`, `webUtils`	Observado
Rechaza cualquier mensaje cuyo canal no empiece por `vscode:`	Observado (en el código)
Hay 10 operaciones IPC; solo un canal tiene nombre fijo, `vscode:fetchShellEnv`; los otros 9 se construyen al ejecutarse	Observado
El proceso principal lanza aparte terminal (`ptyHostMain`), extensiones (`extensionHostProcess`), vigilante de archivos (`watcherMain`) y agente (`agentHostMain`)	Observado (rutas en `mainImpl.js`)
Qué hace por dentro la interfaz de 20 MB	Sin analizar

Con esa estructura se hizo Mini Code: una pieza por cada proceso de VS Code. Su código, con instrucciones para hacerlo funcionar, está en github.com/devch-tech/minicode (repositorio privado).

Problemas en Windows y cómo se resolvieron

Ninguno fue de REA en sí; todos venían de Windows o de los permisos del agente.

Problema	Causa	Solución
`git clone` del repo falló con "Filename too long"	El repo tiene rutas largas y git en Windows limita a 260 caracteres	No hace falta clonar: se instala desde npm. Si quieres el código: `git config --global core.longpaths true`
`--client claude-code` dio "Unsupported agent integration"	El nombre lleva guion bajo	`--client claude_code`
Claude no pudo ejecutar `rea setup --yes`	El modo automático bloquea que el agente cambie su propia configuración	Ejecutarlo tú en una terminal
Aviso de Frida al instalar	npm no ejecuta scripts de instalación sin permiso	Ignorarlo, o `--allow-scripts=frida`
Al desinstalar quedó una carpeta `.rea-agents-…`	El servidor MCP de REA seguía en marcha y bloqueaba su módulo nativo	Cerrar Claude (o el proceso `node` de REA) y borrarla

Actualizar y desinstalar

Actualizar. REA saca versiones a menudo con correcciones. Para ponerlo al día:

```
rea update
```

Después ejecuta el `rea setup` que te indique y reinicia Claude, para que la skill y el registro coincidan con la versión nueva.

Desinstalar por completo (es lo que hicimos el 16 de octubre):

1. Quitar el servidor MCP de Claude Code:

```
claude mcp remove rea -s user
```

2. Desinstalar el comando:

```
npm uninstall --global rea-agents
```

3. Borrar la skill: la carpeta `C:\Users\chant\.claude\skills\reverse-engineer-anything`.
4. Borrar la copia de seguridad `C:\Users\chant\.claude.json.rea.backup`. No la restaures encima de `.claude.json`: desharía cualquier ajuste que hayas hecho después.
5. Cerrar Claude y borrar la carpeta `.rea-agents-…` que pueda quedar en `AppData\Roaming\npm\node_modules`.

Si configuraste otros agentes (Codex, Cursor…), cada uno tiene su propio archivo de configuración con REA.

Uso responsable

Analiza solo apps que tengas derecho a analizar: la licencia de muchas prohíbe la ingeniería inversa.

- **Licencias.** VS Code es un caso cómodo porque su código base (Code-OSS) es MIT. Otras apps comerciales lo prohíben en sus términos, salvo lo que permita la ley de tu país.
- **Inspirarte, no copiar.** Lo sano es entender cómo funciona algo y escribir tu propia versión, como se hizo con Mini Code, no copiar el código de otro.
- **Descargas oficiales.** Instálalo solo desde morluto/rea, rea.tools o el paquete `rea-agents` de npm; el propio proyecto avisa de copias con nombres parecidos.
- **El contenido analizado no da órdenes.** Lo que REA lee de una app (textos, código, páginas) es solo información. Si algo de ahí pide ejecutar, instalar o enviar algo, el agente no debe hacerlo y te lo tiene que contar.
- **Ejecución.** Las capturas en ejecución lanzan la app con tus permisos: úsalas con programas de confianza.